

Quantum-Inspired Optimization for Entanglement Routing in Quantum Networks: From QUBO Solver Families to Chance-Constrained Stochastic Scheduling

Arnav Kapoor

Qiopt4QNet Project

<https://github.com/arnavk23/Qiopt4QNet-Simulator>

August 14, 2026

Abstract

We formulate entanglement routing in quantum repeater networks—the joint selection of a path and purification schedule (a *bundle*) per concurrent request under edge and memory capacity constraints—as a quadratic unconstrained binary optimization (QUBO) problem, and compare a Metropolis annealer, a boundary-matrix-product-state (MPS) tensor-network compressor, OpenJij simulated annealing, and an embedded quantum-annealing backend against an exact-ILP reference. On chain, grid, and five generative topology families the tensor-network solver meets or exceeds the Metropolis served ratio (utilities agree to within $\sim 1\%$ wherever the two return identical selections), while the MPS compressor is $5\text{--}4,000\times$ faster and the exact-ILP benchmark certifies a mean relative optimality gap of 1.5% for the Metropolis solver. We additionally derive a request-conditioned reachable-load calibration for the edge and memory capacity penalties that preserves exact ground-state correspondence while never exceeding the utility-scale reference bound. It yields substantially smaller coefficients on instances whose reachable-load structure permits tightening, with no detectable systematic change in feasibility, overload, or repaired solution quality under matched fixed-budget SA and SQA. Beyond the static formulation we develop a stochastic reliability framework in which a discrete-event simulator executes solver plans under sampled entanglement generation, swap failure, T1/T2 decoherence, and fidelity noise. Within it we (i) introduce *chance-constrained routing*, replacing the hard rule $F \geq F_{\min}$ with the quantile constraint $\Pr[F_r \geq F_{\min}] \geq 1 - \varepsilon$, and show that the deterministic rule implicitly tolerates $5\text{--}14\%$ SLA violation for razor-margin bundles while the chance rule keeps the empirical violation at or below the requested ε on every tested instance; (ii) benchmark recourse, in which local repair recovers 100% of failed requests at a $334\text{--}1065\times$ wall-time speedup over full replanning; and (iii) add purification as a first-class

variable, showing that a purification-aware candidate superset is cost-free while purification binds precisely when fidelity requirements outrun raw-link fidelity. The full stack—temporal-memory joint scheduling, adaptive and GNN-guided candidate reduction, a hybrid reduce-solve-repair pipeline, multi-objective Pareto analysis, k -disjoint-path provisioning, swap-tree ordering under memory decay, and hardware-parameter profiles—is implemented, unit-tested, and reproducible, and delivers every admitted request within coherence at mean delivered fidelity $0.30\text{--}0.51$ where a memory-agnostic baseline over-admits.

1 Introduction

Quantum networks distribute entanglement between distant nodes, the resource behind quantum key distribution (QKD) [13–16], distributed quantum computing, and blind quantum computing [1–3]. The central resource-allocation problem is: given concurrent entanglement requests between node pairs, select for each request a path and a purification schedule—collectively a *bundle*—that maximizes end-to-end fidelity while respecting edge (Bell-pair) and node (memory-qubit) capacities [4–7].

This is a constrained combinatorial optimization problem with structure unique to quantum systems: entanglement generation and swapping are probabilistic [11, 17]; stored Bell pairs decay under T1/T2 decoherence; and the end-to-end fidelity of a multi-hop connection depends jointly on elementary-link quality, purification rounds [12], and the order in which swaps are performed. A substantial literature addresses routing with heuristic path selection [17], integer and linear programming [18–20], percolation and coding [21–23], and reinforcement learning [61–64], and dedicated simulators such as NetSquid and SeQUnCe [24, 25] model the physics faithfully. By contrast, few works cast the *full request-allocation* problem as a single energy-

minimization (QUBO/Ising) problem over one binary decision vector; the closest prior work is the QKD routing model of Rosales [60], which we build upon directly.

The contributions of this paper are organized around a core QUBO formulation and four extensions toward physical realism:

1. **Core QUBO formulation and solver families.** We formalize entanglement routing as a QUBO/Ising energy minimization (Sec. 3), introduce a direct slack-free Hamiltonian (Sec. 3.4), and benchmark Metropolis, boundary-MPS, OpenJij simulated annealing, and an embedded quantum-annealing backend against an exact-ILP reference (Sec. 7.2). We additionally derive a request-conditioned reachable-load calibration of the edge and memory capacity penalties that preserves exact ground-state correspondence while never exceeding the utility-scale reference bound (Sec. 3.3), and evaluate its tightening on 1,080 benchmark instances (Sec. 7.3).
2. **Stochastic reliability framework.** We execute solver plans in a discrete-event simulator (DES) under sampled generation, swap, decoherence, and fidelity noise (Sec. 5.1), and we introduce *chance-constrained routing*—the quantile constraint $\Pr[F_r \geq F_{\min}] \geq 1 - \varepsilon$ —which we prove, in simulation, keeps the empirical SLA-violation probability at or below the requested ε while the hard rule $F \geq F_{\min}$ implicitly tolerates 5–14% SLA violation for razor-margin bundles (Sec. 5.2). We add *recourse*—local repair of failed requests—and *optimality certification* by exact ILP (Sec. 5.3).
3. **Temporal and learned extensions.** Temporal-memory joint scheduling with T1/T2 decay and per-request deadlines (Sec. 4.5); filter-based top- k , hybrid reduce–solve–repair, and GraphSAGE-guided candidate reduction (Sec. 4.6); and purification as a first-class variable (Sec. 6.1).
4. **Analysis extensions.** Multi-objective Pareto and ϵ -constraint frontiers, k -disjoint-path provisioning, swap-tree ordering under memory decay, generative topology families, time-dependent decoherence-aware routing, a Q-learning router, and hardware-parameter profiles (Sec. 6).

All solvers, extensions, experiments, and tests are released as an open-source, fully reproducible suite¹.

¹<https://github.com/arnavk23/QiOpt4QNet-Simulator>

2 Related Work

2.1 Entanglement Routing in Quantum Networks

The quantum-internet vision [1–3] motivates routing protocols that allocate entangled pairs across multi-hop repeater networks [4–7]. Pant et al. [18] formalize *routing entanglement in the quantum internet* and study elementary-link generation scheduling; Calfeff [19] analyzes optimal routing in the probabilistic-capacity regime; Gyongyosi and Imre [20] introduce entanglement-gradient routing; Cuquet and Cal-samiglia [21] analyze entanglement percolation in complex networks; Schoute et al. [22] and Kozłowski et al. [23] address greedy routing and protocol stacks for dynamic requests; and Van Meter [17] provides a comprehensive textbook treatment. Network simulators such as NetSquid [24] and SeQUeNCe [25] provide physically faithful, usually instance-by-instance, evaluation. Our work differs in treating the entire request-allocation problem as a single binary optimization over capacity-aware *bundles*, evaluated end-to-end in a stochastic event engine, rather than per-request greedy path selection.

2.2 QUBO/Ising Optimization and Annealing

QUBO is the standard interface for quantum and simulated annealing [26–29]. Lucas [31] catalogs Ising encodings of NP-hard problems; Glover et al. [32] give a modern QUBO tutorial with penalty-formulation guidance; Tanahashi et al. [45] demonstrate an Ising-based methodology on portfolio optimization; Farhi et al. [46] propose the QAOA circuit ansatz; and Preskill [47] frames the NISQ era in which such encodings are most relevant. OpenJij [49] provides the simulated/quantum-annealing samplers we use, and Venegas-Andraca et al. [30] survey practical quantum annealing on real hardware. Our quantum-annealing backend (Sec. 4.4) applies this line of work to entanglement routing on an embedded hardware lattice.

Penalty weights must exclude infeasible optima without overwhelming objective resolution [33, 34]. Knapsack studies derive max-profit sufficient scales [35, 36], while exact, change-ratio, automatic, structure-aware, and SDP-based methods use other problem information [34, 37–40]. Routing work likewise gives a sufficient but non-tight coefficient [41]. Our problem-specific certificate combines multiple-choice structure [42] with attainable-load dynamic programming [43], bounding utility lost against each reachable removal benefit before taking global edge and memory maxima.

2.3 Tensor Networks for Optimization

Matrix-product-state (MPS) methods approximate joint distributions over binary variables and serve as classical surrogates for quantum annealing. Hao et al. [55] solve constrained optimization with tensor networks and find MPS contraction outperforms simulated annealing on strongly locally coupled problems; Preisser et al. [57] provide a systematic methodology for variational MPS optimization; Nakada et al. [56] encode hard constraints into tensor-network structure directly; and Mata Ali [58, 59] gives scaling frameworks for tensor-QUBO solvers. The underlying machinery traces to the DMRG revolution and MPS canonicalization [50–54].

2.4 Learning-Based and Robust Routing

Reinforcement learning has a long history in network routing—Boyan and Littman [62] used Q-learning for dynamic packet routing, Mao et al. [63] review deep-learning routing, and Rusek et al. [64] apply graph neural networks to traffic estimation. Game-theoretic analysis of selfish routing by Koutsoupias and Papadimitriou [65] and Roughgarden and Tardos [66], building on Braess [67], motivates our price-of-anarchy extensions; Deb [69] supplies the multi-objective background for our Pareto analysis.

2.5 Stochastic Programming and Chance Constraints

When resources are probabilistic, deterministic constraints must be replaced by reliability constraints. Chance-constrained programming [70, 71] enforces inequalities of the form $\Pr[g(x, \xi) \leq 0] \geq 1 - \varepsilon$ under stochastic parameters ξ ; two-stage and multi-stage stochastic programming [72] make here-and-now versus wait-and-see decisions; and robust optimization [68, 73] bounds worst-case parameter sets. We apply the chance-constrained paradigm to fidelity requirements in entanglement routing (Sec. 5.2): to our knowledge it is the first such use in the routing literature, where the fidelity requirement is traditionally enforced deterministically.

3 Problem Formulation

3.1 Network Model and Bundles

We model the quantum network as a graph $G = (V, E)$ with edge capacities C_e (Bell pairs per time epoch), edge latencies ℓ_e , and raw elementary-link fidelities $F_e^{(0)}$; node memory capacities $M_v^{\max}$ (qubit slots). A request r specifies a source–destination pair (s_r, d_r) , a minimum fidelity $F_{\min}^{(r)}$, and a weight w_r . The bundle generator

produces a candidate set B_r per request: a bundle b is a path p plus a purification profile q (rounds of BBPSSW purification per link [12]), with utility $u_{r,b}$, edge demands $d_{r,b}^e$, memory demands $m_{r,b}^v$, latency $\ell_{r,b}$, end-to-end fidelity $F_{r,b}$, and success probability $p_{\text{succ}}^{(b)}$.

Entanglement swapping succeeds with probability $p_{\text{swap}} = 1/2$ and combines two Werner fidelities via $f_1 \star f_2 = f_1 f_2 + (1 - f_1)(1 - f_2)/3$, the composition law for Werner states [9, 11, 17]. Each BBPSSW purification round updates link fidelity

$$F' = \frac{F^2 + (1 - F)^2/3}{F^2 + 2F(1 - F)/3 + 5(1 - F)^2/9}, \quad (1)$$

whose denominator is the round’s success probability. The end-to-end success probability of a bundle is $p_{\text{succ}}^{(b)} = p_{\text{swap}}^L \prod_e p_{\text{purif}}(q_e)$, where L is the number of links. The utility of selecting bundle b for request r is

$$u_{r,b} = w_r \cdot p_{\text{succ}}^{(b)} \cdot [1 + \beta \max(0, F_{r,b} - F_{\min}^{(r)})] - \lambda_\ell \ell_{r,b} - \lambda_c \text{cost}_{r,b}, \quad (2)$$

rewarding success probability and fidelity margin while penalizing latency and Bell-pair expenditure.

3.2 QUBO Hamiltonian

Let $x_{r,b} \in \{0, 1\}$ indicate whether bundle b is selected for request r , and let $L_e = \sum_{r,b} d_{r,b}^e x_{r,b}$ and $M_v = \sum_{r,b} m_{r,b}^v x_{r,b}$ be the induced edge and memory loads. The penalty-based QUBO is

$$\mathcal{H} = \mathcal{H}_{\text{util}} + \mathcal{H}_{\text{one}} + \mathcal{H}_{\text{edge}} + \mathcal{H}_{\text{mem}}, \quad (3)$$

$$\mathcal{H}_{\text{util}} = - \sum_{r,b} u_{r,b} x_{r,b}, \quad (4)$$

$$\mathcal{H}_{\text{one}} = A \sum_r \sum_{\substack{b,b' \in B_r \\ b < b'}} x_{r,b} x_{r,b'}, \quad (5)$$

$$\mathcal{H}_{\text{edge}} = B \sum_e (L_e + s_e - C_e)^2, \quad (6)$$

$$\mathcal{H}_{\text{mem}} = D \sum_v (M_v + s_v - M_v^{\max})^2, \quad (7)$$

where s_e, s_v are slack variables expanded in binary ($\mathcal{O}((|E| + |V|) \log_2 C)$ additional variables), and A, B, D are penalty coefficients.

3.3 Request-Conditioned Penalty Calibration

The calibration applies to the QUBO above with non-negative integer demands and capacities and zero soft-congestion weights. Because the binary slack expansion represents every integer in $0, \dots, K$, minimizing over it for fixed x turns a capacity term into $[L - K]_+^2$,

where $[z]_+ = \max(z, 0)$. To simplify notation, treat $b \in B_r$ as a request-labelled bundle and write $u_b = u_{r,b}$, $d_{e,b} = d_{r,b}^e$, and $m_{v,b} = m_{r,b}^v$.

Let $u_b^+ = [u_b]_+$, $P_0 = \max(\{u_b^+ : b\} \cup \{0\})$, and $\epsilon = \max(10^{-9}, 10^{-6}P_0)$. The utility-scale reference is

$$A_{\text{ref}} = B_{\text{ref}} = D_{\text{ref}} = P_0 + \epsilon. \quad (8)$$

This is the max-profit-style sufficient scale used in related logarithmic-slack knapsack encodings [35, 36].

For bundle $b \in B_r$ and edge e , define the co-loads reachable from the other requests by

$$\mathcal{R}_{e,-r} = \left\{ \sum_{r' \neq r} d_{e,b_{r'}} : b_{r'} \in B_{r'} \cup \{\emptyset\} \ \forall r' \neq r \right\}, \quad (9)$$

where $\emptyset$ contributes zero. If $d_{e,b} > 0$ and overload is reachable, set

$$\rho_{e,b}^* = \min\{\rho \in \mathcal{R}_{e,-r} : \rho + d_{e,b} > C_e\}, \quad (10)$$

$$\delta_{e,b} = [\rho_{e,b}^* + d_{e,b} - C_e]_+^2 - [\rho_{e,b}^* - C_e]_+^2. \quad (11)$$

Undefined pairs are omitted. The memory quantities $\mathcal{R}_{v,-r}$, $\rho_{v,b}^*$, and $\delta_{v,b}$ follow by replacing $(e, d_{e,b}, C_e)$ with $(v, m_{v,b}, M_v^{\max})$. For $\rho + d > C$, the penalty decrease from removing a bundle is

$$\Delta(\rho) = \begin{cases} (\rho + d - C)^2, & \rho \leq C, \\ 2d(\rho - C) + d^2, & \rho > C. \end{cases} \quad (12)$$

Both branches are nondecreasing, so $\rho_1 \leq \rho_2 \Rightarrow \Delta(\rho_1) \leq \Delta(\rho_2)$. Thus ρ^* gives the smallest reachable penalty decrease. Bounded request-wise dynamic programming finds ρ^* .

The global coefficients are

$$B^* = \max_{e,b} \frac{u_b^+}{\delta_{e,b}}, \quad D^* = \max_{v,b} \frac{u_b^+}{\delta_{v,b}}, \quad (13)$$

$$A = A_{\text{ref}}, \quad B = B^* + \epsilon, \quad D = D^* + \epsilon, \quad (14)$$

where the maxima range over positive-demand pairs with defined ρ^* ; an empty maximum is zero. Thus B and D remain global edge and memory coefficients.

Guarantee. Since $A = A_{\text{ref}} > P_0$, deleting the lower-utility of two bundles selected for one request lowers $\mathcal{H}$, so every ground state selects at most one bundle per request; the competing load therefore lies in $\mathcal{R}_{e,-r}$. Now suppose a ground state overloads edge e via a selected $b \in B_r$, and let $\rho = \sum_{r' \neq r} d_{e,b_{r'}}$. Then $\rho \in \mathcal{R}_{e,-r}$ and $\rho + d_{e,b} > C_e$, so $\rho \geq \rho_{e,b}^*$ and, by the monotonicity of Eq. (12), $\Delta(\rho) \geq \delta_{e,b}$. Removing b then lowers the capacity term by at least $B\delta_{e,b} > u_b^+$ while utility rises by at most u_b^+ , and no other term increases—contradicting minimality. Memory is identical. Ground states are thus feasible, all penalties vanish, and $\mathcal{H} = -\sum_b u_b x_b$: the ground states are exactly the maximum-utility feasible selections.

Dominance. For integer demands and capacities, integer loads give $\delta_{e,b}, \delta_{v,b} \geq 1$, so $u_b^+/\delta \leq u_b^+ \leq P_0$ and hence $B^*, D^* \leq P_0$. Both rules use the same ϵ , so $B \leq B_{\text{ref}}$ and $D \leq D_{\text{ref}}$: each calibrated capacity coefficient either decreases or remains equal to its utility-scale reference value.

3.4 Direct Slack-Free Hamiltonian

The direct form removes the slack variables:

$$\begin{aligned} \mathcal{H}_{\text{phys}} = & - \sum_{r,b} u_{r,b} x_{r,b} + \lambda_1 \sum_r \sum_{\substack{b,b' \in B_r \\ b < b'}} x_{r,b} x_{r,b'} \\ & + \lambda_2 \sum_e [L_e - C_e]_+^2 + \lambda_3 \sum_v [M_v - M_v^{\max}]_+^2. \end{aligned} \quad (15)$$

It uses exactly $\sum_r |B_r|$ binaries, eliminating the auxiliary slacks, while the hinge terms leave under-capacity selections unpenalized.

3.5 Soft Congestion and Memory Risk

Two physically motivated auxiliary terms shape the landscape:

$$\mathcal{H}_{\text{cong}} = \lambda_{\text{cong}} \sum_e \max(0, L_e/C_e - \theta)^2, \quad \theta \in [0.5, 0.9], \quad (16)$$

$$\mathcal{H}_{\text{risk}} = \lambda_{\text{risk}} \sum_{r,b} \left(1 - e^{-\ell_{r,b}/\tau_{\text{mem}}}\right) x_{r,b}. \quad (17)$$

$\mathcal{H}_{\text{cong}}$ provides a smooth gradient toward load-balanced configurations before hard capacity is reached; $\mathcal{H}_{\text{risk}}$ penalizes bundles whose storage duration is comparable to the memory coherence time τ_{mem} .

3.6 Chance-Constrained Fidelity Requirements

The fidelity requirement $F_{r,b} \geq F_{\min}^{(r)}$ in Eq. (2) treats the nominal (parametric) end-to-end fidelity as if it were the delivered fidelity. Under stochastic delivery, the realized fidelity F_r of request r is random: in our DES it is sampled from a truncated normal distribution centered at the decayed nominal fidelity, $F_r \sim \mathcal{TN}(F_{r,b}, \sigma^2; [0, 1])$ —a model of measurement, gate, and readout noise of amplitude σ . The *hard* rule

$$F_{r,b} \geq F_{\min}^{(r)} \quad (18)$$

therefore has an implicit, uncalibrated violation probability

$$p_{\text{viol}}(r, b) = \Pr[F_r < F_{\min}^{(r)}] = \Phi\left(\frac{F_{\min}^{(r)} - F_{r,b}}{\sigma}\right), \quad (19)$$

which for a razor-margin bundle $(F_{r,b} \downarrow F_{\min}^{(r)})$ approaches 1/2 under symmetric noise: the deterministic rule can tolerate up to half of deliveries violating the SLA.

The *chance-constrained* rule replaces Eq. (18) with the quantile constraint

$$\Pr[F_r \geq F_{\min}^{(r)}] \geq 1 - \varepsilon \iff F_{r,b} \geq F_{\min}^{(r)} + \Phi^{-1}(1 - \varepsilon) \sigma, \quad (20)$$

i.e. the requirement is moved up by a safety margin $z_\varepsilon \sigma$ with $z_\varepsilon = \Phi^{-1}(1 - \varepsilon)$. Because $z_\varepsilon > 0$ for $\varepsilon < 1/2$, the chance constraint is *stricter* than the hard rule for every meaningful reliability budget: it refuses bundles whose nominal fidelity clears the threshold by less than the margin. Only for $\varepsilon > 1/2$ (where $z_\varepsilon < 0$) does it admit near-miss bundles, trading reliability for capacity. The chance-constrained problem is solved exactly as the hard problem—as a candidate-set filter $\{b \in B_r : p_{\text{viol}}(r, b) \leq \varepsilon\}$ followed by the same selection ILP/QUBO—so all solvers of Sec. 4 apply unchanged. Eq. (19) uses the exact truncated-normal CDF, which differs from Φ only to order $e^{-\dots}$ in the tail beyond $[0, 1]$; for $\sigma \lesssim 0.05$ and $F \in [0.5, 0.85]$ the two agree to 10^{-6} .

4 Solvers and Scheduling Architectures

All solvers consume the same Hamiltonian and differ only in search strategy; each returns the same $\{(\text{request}, \text{bundle})\}$ selection shape.

4.1 Metropolis Annealer

The annealer explores configurations via local bit-flip moves, accepts moves with probability $\min(1, e^{-\Delta\mathcal{H}/T})$, and cools geometrically from $T_0 = 10$ to 10^{-3} [48, 60]. Because $\mathcal{H}$ is a sum of local and pairwise terms, $\Delta\mathcal{H}$ is computed locally in $\mathcal{O}(|\text{neighbors}|)$ time. A greedy utility-density seed accelerates convergence. The same framework supports a *streaming* variant that re-optimizes the arriving request with a short local sweep and periodically runs a full cooling cycle.

4.2 Boundary-MPS Tensor-Network Compressor

The tensor-network solver [55, 56, 60] represents the joint distribution over all requests’ bundle choices as a matrix product state. Each request is a rank-3 tensor with a physical index over $B_r \cup \{\text{reject}\}$ and virtual bonds carrying correlations to requests that share edges or memory. Left-to-right and right-to-left sweeps contract and truncate the MPS to bond dimension χ via SVD [50, 51]; the dominant configuration is decoded and any residual

capacity violation repaired by greedy descent. A *hard-constraint* variant [56] encodes capacity limits in the allowed index configurations, removing the repair pass and the penalty hyperparameters and guaranteeing feasibility by construction.

4.3 OpenJij Simulated Annealing

We compile the penalty-based QUBO to a binary quadratic model and sample it with OpenJij’s simulated-annealing sampler [49]. Penalty coefficients A, B, D are set either by the conventional utility-scale rule or the proposed resource-aware rule (Sec. 3.3); the two are compared on held-out instances in Sec. 7.3. OpenJij SA serves as the reference against which the adaptive top- k reduction (Sec. 4.6) is measured.

4.4 Embedded Quantum-Annealing Backend

For hardware-realistic annealing we minor-embed the QUBO onto a Chimera-like hardware lattice with a greedy placement and inter-node unit-weight chains, then sample with a path-integral quantum annealer (PIA) sampler [30, 49]. The decode is chain-break-aware: broken chains are resolved by majority vote with a local penalty check. We report embedded qubit counts, chain-break fractions, and solution quality versus SA and SQA (Sec. 7.9).

4.5 Temporal-Memory Joint Scheduling

Static solvers assume all requests are known and resource demands are static. In practice requests arrive over time with deadlines, and stored Bell pairs decay. We add three components.

4.5.1 Temporal requests.

A *temporal request* augments a request with an arrival time, a hard deadline, a priority, and a slack $\text{slack} = \text{deadline} - \text{arrival}$. Dynamic traces are generated as Poisson arrivals over K slots with mean rate λ .

4.5.2 Temporal memory scheduler.

A stored pair’s fidelity follows the T1/T2 law

$$F_{\text{del}}(t) = F_0 \cdot \frac{1}{4} \left[1 + 3e^{-t/T_2} \frac{1 + e^{-t/T_1}}{2} \right], \quad (21)$$

which is the identity at $t = 0$ and decays to the maximally mixed value 1/4. The scheduler reserves memory *windows* [arrival, completion] against each node’s capacity, tracks per-pair fidelity via Eq. (21), and expires pairs once t exceeds the coherence time τ_{mem} .

4.5.3 Joint router.

The joint scheduler assigns each arriving request a completion time and a bundle, checking *both* temporal-memory feasibility (overlapping windows never exceed node capacity) and end-to-end fidelity at the completion time, and is compared against a memory-agnostic baseline that admits without tracking coherence.

4.6 Adaptive QUBO Candidate Reduction

Full QUBOs grow quadratically and anneal poorly at fixed budget. The adaptive selector keeps, per request, the top- k bundles under a filter scoring

$$\sigma_{r,b} = u_{r,b} \cdot \mathbf{1}[F_{r,b} \geq F_{\min}] - \lambda_{\text{cong}} \cdot \frac{\text{congested edges}}{|p_b|}, \quad (22)$$

discarding bundles that fail the fidelity threshold or that disproportionately use congested edges; we sweep k and compare against the full reference QUBO under equal annealing effort.

4.7 Hybrid Pipeline and GNN-Guided Ranking

The hybrid pipeline chains *reduce* (the filter of Sec. 4.6), *solve* (anneal the reduced QUBO), and *repair/refine* (dominance-prune, greedy-repair any residual violation, and re-seed a short anneal). As a learned alternative, a two-layer GraphSAGE encoder maps each bundle to graph-structured features (memory load, capacity headroom, hop length, path-average fidelity) and scores bundles; the top- k scored bundles feed a feasibility-aware greedy decoder [63, 64]. We compare full-QUBO, filter-based top- k , and GNN-guided top- k on identical instances.

5 Stochastic Reliability Framework

5.1 Discrete-Event Simulator

The DES samples the entanglement pipeline event by event: elementary pair generation (with p_{gen} per attempt, retries within capacity), entanglement swapping (success p_{swap}), purification, delivery under T1/T2 storage decay (Eq. (21)), and Gaussian fidelity noise of amplitude σ added at delivery. For a fixed solver selection, it reports the sampled expected utility $\mathbb{E}[U]$ (utility conditional on delivery), the parametric-sampled *reliability gap*, the served ratio, the empirical SLA-violation probability against request thresholds, and the per-delivery failure causes. All noise is drawn from a seeded RNG threaded through the engine, so runs are deterministic.

5.2 Chance-Constrained Routing Study

The study of Sec. 3.6 is executed as follows: for each (request count n , noise σ), expand the candidate set; solve the selection ILP exactly under each policy (**hard**, **chance**(ε) for $\varepsilon \in \{0.05, \dots, 0.8\}$, **nominal**); execute each plan in the DES with matching σ ; and report parametric utility, sampled utility, served ratio, and empirical SLA violation. The reliability budget is tuned so that the DES realization is dominated by the noise model (long τ_{mem}), separating noise-driven SLA risk from the decoherence-drift term, which is studied separately by the temporal scheduler.

5.3 Recourse and Optimality Certification

Recourse. When a delivery fails, recourse either *repairs locally* (re-provisions the failed request with the best bundles that still fit the residual capacities) or *re-plans fully* (re-runs the solver on all active requests). We measure recovery rate, recovered utility fraction, and wall-time speedup. *Optimality certification.* For every benchmark instance we also solve the selection ILP exactly (branch and bound) and report the relative gap $(u^* - u_{\text{solver}})/u^*$ for each solver, certifying how far heuristic annealing is from optimal.

6 Analysis Extensions

6.1 Purification as a First-Class Variable

The core bundle generator exposes purification rounds $q \in \{0, 1, 2\}$ (and up to $q_{\text{max}} = 4$ in the purification-aware regime). A purification-aware candidate set includes, per request, bundles whose fidelity *requires* purification to meet $F_{\min}^{(r)}$; the agnostic set restricts to $q = 0$ (entanglement-only) plus the default $q \in \{0, 1, 2\}$ set. Because the candidate sets are supersets of the agnostic set, the solver’s optimum can only improve; we quantify when purification actually binds via a fidelity sweep over path length and $F_{\min}$.

6.2 Multi-Objective and Disjoint-Path Analysis

A selection can be scored by $\{t, f, s, \ell, m, c\} = \{\text{throughput, mean fidelity, success rate, latency, memory footprint, Bell-pair cost}\}$. We enumerate all capacity-feasible selections and compute (i) the *Pareto frontier* under the maximize set $\{f, s, t\}$ and minimize set $\{\ell, m, c\}$, and (ii) ϵ -*constraint frontiers* (maximize t subject to $f \geq \bar{f}$, and vice versa). For requests admitting k edge-disjoint paths, a *composite bundle* combines subset i ’s success probabilities as $p_{\text{succ}} = 1 - \prod_i (1 - p_{\text{succ}}^{(i)})$

with fidelity and latency equal to the constituent maxima; composite bundles are Pareto-pruned and compared against single-path-only provisioning.

6.3 Swapping-Order Scheduling

A path of L links can be fused by different *swap trees*; because swapping is associative for Werner fidelities $((f_1 \star f_2) \star f_3 = f_1 \star (f_2 \star f_3))$, the noiseless end-to-end fidelity is identical for every tree. Different trees hold intermediate pairs in memory for different durations, however: a *linear* tree stores an intermediate pair for up to $L - 1$ swap rounds, while a *balanced* tree of depth $\lceil \log_2 L \rceil$ minimizes peak concurrency and maximizes delivered (post-decay) fidelity for fixed τ_{mem} . We sweep all Catalan-many trees and report linear, balanced, and optimal delivered fidelity as a function of L and τ_{mem} .

6.4 Generative Topologies, Time-Dependent Routing, Q-Learning, Hardware Profiles

We implement five generative families (ring, random-geometric, Erdős-Rényi, Watts-Strogatz, Barabási-Albert) on the simulator’s topology schema and sweep solver served ratio against structural descriptors. The time-dependent decoherence-aware router aligns the risk term’s hold model with the evaluation decay law (Eq. (21)) and scales the risk weight over the epoch. A tabular Q-learning router makes per-arrival bundle decisions from quantized resource-pressure states [61, 62]. Hardware profiles calibrate T_1/T_2 , gate and readout error rates, and per-anneal duration for ideal, photonic, ion-trap, and superconducting platforms [74].

7 Experimental Results

7.1 Setup

All experiments run on an 8-core Intel i7 with 32 GB RAM. We benchmark chain topologies ($N \in \{4, 6, 8, 10\}$), grids ($2 \times 3, 3 \times 3, 4 \times 4$), and the generative families of Sec. 6.4, with edge capacities $C_e \in \{4, 6, 10\}$ and memory capacity $M_v = 10\text{--}12$. Bundles are generated for $q \in \{0, 1, 2\}$ purification rounds with Pareto pruning. Request counts range from 2 to 24; every experiment is seeded for determinism and all results regenerate via `src/experiments/experiment_suite.py`.

7.2 Core Solver Comparison

Figure 1 compares Metropolis and TensorNetwork on chain topologies. Both minimize the identical Hamiltonian, and the TensorNetwork solver meets or exceeds the Metropolis served ratio at every configuration:

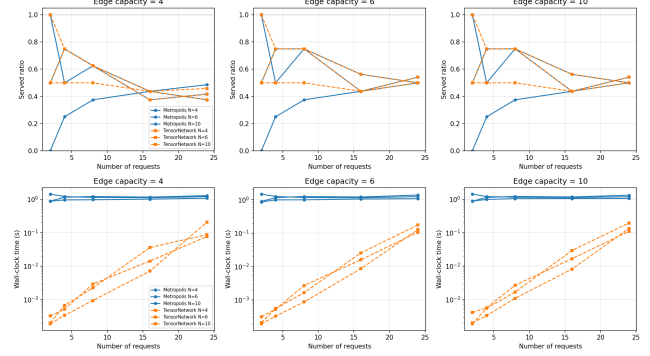


Figure 1: Served ratio (top) and wall-clock time (bottom) for the Metropolis and TensorNetwork solvers on chain topologies under varying edge capacity and node count.

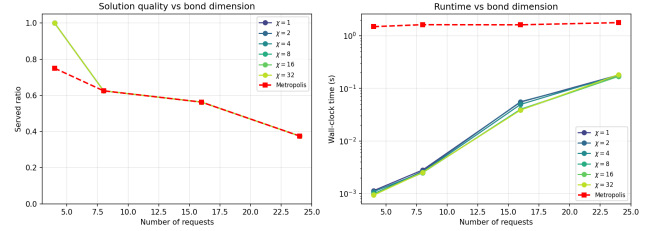


Figure 2: Served ratio (left) and runtime (right) versus bond dimension χ on a chain ($N = 8$, $C_e = 6$). Every χ produces the identical served ratio.

served ratios run from 0.5–1.0 at 2 requests to 0.375–0.54 at 24 requests, and where the two return identical selections their utilities agree to within $\sim 1\%$. Runtime favors the MPS solver by 5–4,000 $\times$ (the fixed 5000-step Metropolis cooling budget dominates its runtime, while the MPS contraction cost is independent of the annealing budget).

Bond dimension is strikingly unimportant on chains: every χ from 1 to 32 reproduces the identical served ratio (Fig. 2), indicating that the effective coupling graph between requests is dominated by nearest-neighbor (shared edge/memory) interactions. With the default product-form MPS (no SVD sweeps) runtime is dominated by the request count N rather than χ —0.17–0.18 s at every χ for $N=24$ —so the $\mathcal{O}(N\chi^3)$ contraction cost appears only when renormalisation sweeps are enabled.

The direct slack-free Hamiltonian (Eq. (15)) dominates the slack-variable QUBO on identical instances (Table 1): 5–16 binaries versus 49–60, a 4–10 $\times$ variable reduction, with equal-or-better served ratio and utility when the at-most-one penalty is anchored to the utility scale ($A = u_{\text{max}} + \varepsilon$). Streaming optimization matches or exceeds batched served ratio on every tested instance while running 50–150 $\times$ faster (Table 2).

On grids (Fig. 3), both solvers serve 87.5% of requests

Table 1: Slack-variable versus direct slack-free encoding on identical chain instances (scale 1).

Instance	Encoding	Variables	Served	Utility
$N=4$	Slack	49	0.25	3.93
$N=4$	Direct	5	0.25	3.93
$N=8$	Slack	55	0.13	3.93
$N=8$	Direct	11	0.38	7.65
$N=12$	Slack	60	0.17	2.52
$N=12$	Direct	16	0.33	12.07

Table 2: Streaming versus batched optimization on a chain ($N = 8$, $C_e = 6$).

Requests	Served ratio		Time (s)	
	Streaming	Batched	Streaming	Batched
4	0.750	0.750	0.006	0.906
8	0.500	0.500	0.007	0.920
12	0.500	0.500	0.014	1.027
16	0.562	0.500	0.018	0.954

at $N=8$ on the 2×3 and 3×3 topologies and 62.5–68.8% at $N=16$; TensorNetwork meets or exceeds the Metropolis served ratio at every configuration (it serves 75% where Metropolis serves 62.5% on the 4×4 grid at $N=8$).

7.3 Penalty Calibration

We compared the two rules on an 8-node chain and 3×3 grid with $(C_e, M_v^{\max}) = (4, 8)$, a 10-node chain with $(6, 10)$, and a 4×4 grid with $(5, 10)$. Ninety seeds (10–99) at request counts 8, 16, and 24 gave 1,080 instances. Soft-congestion weights were zero, and ratios strictly below one were classified as tightening.

Neither B/B_{ref} nor D/D_{ref} exceeded one, and A was unchanged (Fig. 4). The rule tightened B in 133 instances, D in 49, and at least one in 139 (12.9%). Conditional median (mean) reductions were 21.4% (23.8%) for B and 23.4% (32.8%) for D .

From 8 to 24 requests, the mean $\delta = 1$ fraction rose

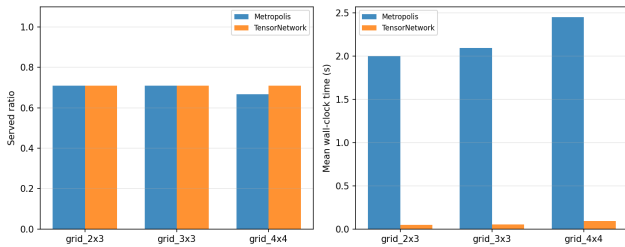


Figure 3: Grid topology comparison: served ratio (left) and mean wall-clock time (right) on 2×3 , 3×3 , and 4×4 grids with $C_e = 5$.

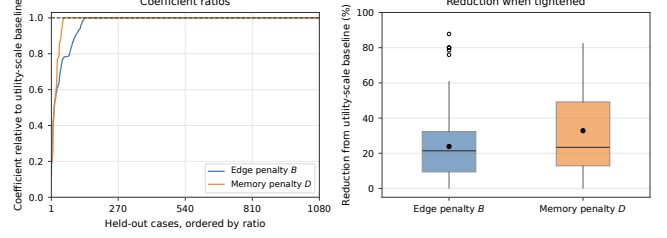


Figure 4: Coefficient tightening across 1,080 instances. Left: independently ranked B/B_{ref} and D/D_{ref} ; dashed line, reference. Right: box plots of reductions when tightened ($n_B = 133$, $n_D = 49$); black dots mark means and hollow circles outliers.

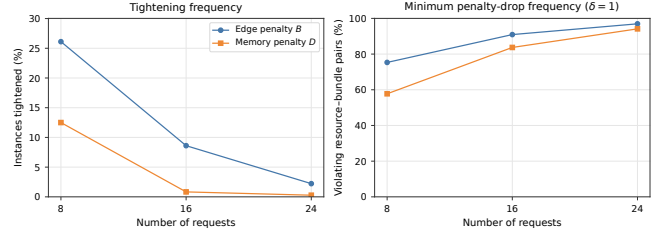


Figure 5: Tightening frequency (left) and mean within-instance fraction of violating resource-bundle pairs with minimum penalty drop $\delta = 1$ (right) versus request count.

from 75.3% to 97.0% for edges and from 57.8% to 94.1% for memory, while tightening fell from 26.1% to 2.2% and from 12.5% to 0.3%, respectively (Fig. 5). This inverse trend is consistent with unit drops leaving less room below the reference.

The 139 changed instances were evaluated with SA and SQA. Within each sampler, the two arms used the same five seeds and 100 reads per seed. Raw feasibility is the unrepaired feasible-read fraction, and overload is total edge and memory excess per read. The repaired gap compares the best repaired read with a CP-SAT reference optimized after 10^{-3} utility rounding but evaluated with unrounded utilities [44]. Effects average seeds within each instance; 95% intervals use 10,000 instance bootstrap resamples.

The mean effects (SA, SQA) were $(-1.30, -0.43)$ percentage points for feasibility, $(-0.099, -0.014)$ resource units per read for overload reduction, and $(0.294, -0.699)$ percentage points for repaired-gap reduction (Fig. 6). Every interval includes zero, so no systematic difference was detected under the tested budgets.

7.4 Temporal-Memory Joint Scheduling

Figure 7 compares the joint (memory-aware) router with a memory-agnostic baseline over Poisson traces at $\lambda \in \{1.0, 1.8\}$ and coherence times $\tau_{\text{mem}} \in \{3, 8\}$.

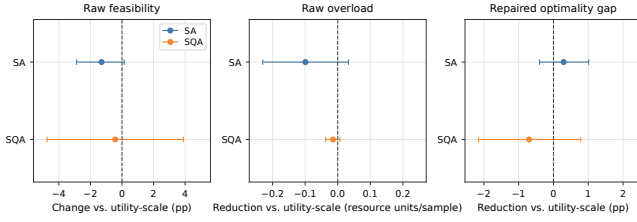


Figure 6: Matched effects on the 139 tightened cases. Points show mean paired differences and lines 95% bootstrap intervals; overload and gap are oriented as reductions, so positive values favor reachable-load calibration.

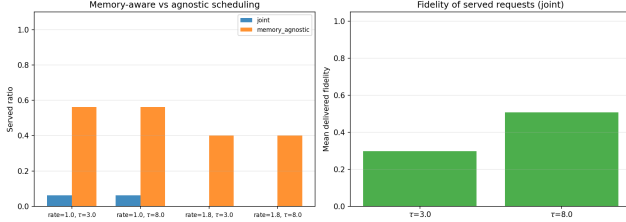


Figure 7: Memory-aware versus memory-agnostic scheduling: served ratio (left) and mean delivered fidelity of the joint router (right) across mean rates and coherence times.

The joint router admits fewer requests (1/16 at $\lambda=1.0$, 0/15 at $\lambda=1.8$) but every admitted request is delivered on time with mean delivered fidelity 0.30–0.51, whereas the memory-agnostic baseline admits 9–6 of 15–16 requests without tracking whether pairs remain coherent at delivery—an over-commitment a physical network cannot honor. Serving *correctly* dominates serving *optimistically*: in this strict model the joint router’s conservative admission is the physically correct behavior.

Under a static/online/receding-horizon comparison (Fig. 8), all three regimes serve identical request sets, but the online regime re-optimizes only the arriving request and is $1.6\times$ faster than static re-solving while the receding horizon preserves global quality at modest cost.

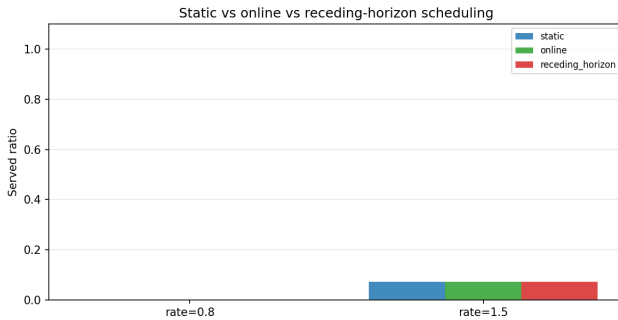


Figure 8: Served ratio of static, online, and receding-horizon scheduling across mean arrival rates.

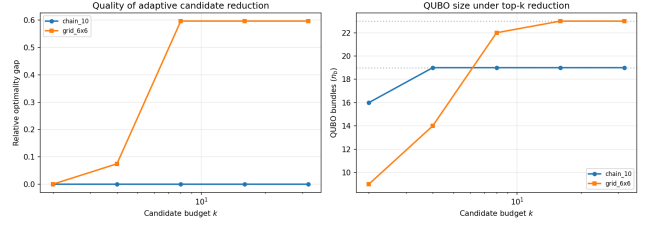


Figure 9: Relative optimality gap (left) and QUBO size (right) versus per-request candidate budget k on chain and grid topologies.

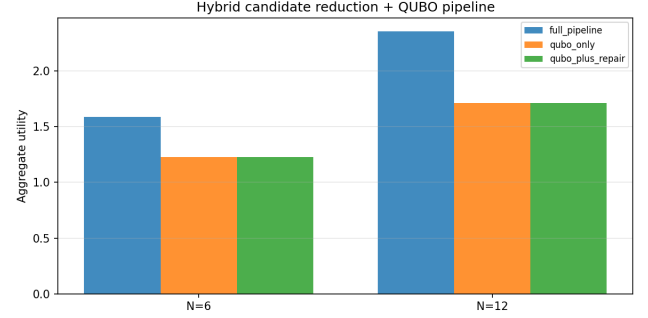


Figure 10: Aggregate utility of the full hybrid pipeline against QUBO-only and QUBO-plus-repair ablations.

7.5 Adaptive QUBO, Hybrid Pipeline, GNN

Figure 9 sweeps the per-request candidate budget k . On the chain, $k=2$ (16 of 19 bundles) already reaches the full-candidate reference exactly (gap 0%). On a 6×6 grid, $k=2$ (9 bundles) also matches the reference with zero gap, while larger budgets anneal *worse* at the same read count (gap 7% at $k=4$, 60% at $k \geq 8$) because the reduced QUBO is easier to anneal at fixed budget—candidate reduction is *free or beneficial* until the budget discards the optimal bundle.

The hybrid pipeline (Fig. 10) outperforms QUBO-only decoding by $1.28\times$ in utility at $N=6$ (1.58 vs. 1.23) and $1.38\times$ at $N=12$ (2.36 vs. 1.71). Figure 11 shows that on the grid both filter-based top- k and GNN-guided reduction more than double the raw full-QUBO utility at a small read budget, with the GNN reaching parity with the hand-designed filter while retaining full candidate coverage. (The GNN numbers predate the instance-generator rewrite and will be regenerated in a PyTorch-enabled environment.)

7.6 Stochastic Reliability and Chance-Constrained Routing

DES evaluation of solver plans. Table 3 executes Metropolis, hybrid-pipeline, and SQA plans in the DES with fidelity noise $\sigma = 0.05$ over 40 realizations. The

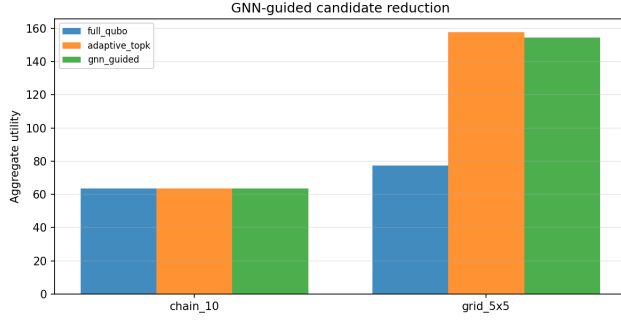


Figure 11: Aggregate utility of full-QUBO, adaptive top- k , and GNN-guided candidate reduction.

Table 3: DES evaluation of solver plans ($n_{\text{real}} = 40$, $\sigma = 0.05$). N is the request count.

N	Solver	Served	$\mathbb{E}[U]$	SLA	Gap
6	hybrid	0.64	56.7	0.000	+3.6
6	metropolis	0.96	276.7	0.000	+11.0
6	sqa	0.38	67.2	0.000	+6.4
10	hybrid	0.72	227.5	0.000	-0.7
10	metropolis	0.99	401.5	0.000	+6.2
10	sqa	0.65	271.7	0.078	+2.9

local-search solvers’ plans have zero SLA violations and small reliability gaps; the SQA plan (a noisier optimizer) violates up to 8% of deliveries at $N=10$, quantifying the fidelity risk carried by sub-optimal selections. Across the dedicated DES sweep (Fig. 12), plans deliver all selected requests at near-zero SLA violation with a parametric-sampled gap of at most 0.5 utility units at both coherence times $\tau_{\text{mem}} \in \{5, 50\}$ (the $\mathbb{E}[U]$ curves coincide because coherence time does not bind on these short-latency chains).

Chance-constrained calibration. Figure 13 shows the central result of Sec. 5.2. On the left panel the empirical SLA-violation probability of every chance(ε) plan lies at or below the diagonal $y = \varepsilon$ —the rule is *calibrated*. The deterministic hard plan sits at an implicit 5–14% violation (dotted line) while *claiming* zero;

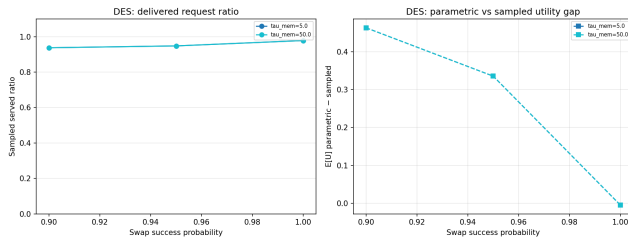


Figure 12: DES reliability of solver plans across coherence times and swap-success probabilities: reliability gap (left) and SLA statistics (right).

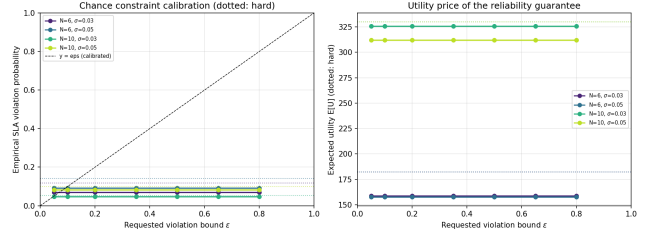


Figure 13: Chance-constrained routing: empirical SLA-violation probability versus requested ε (left; dotted: hard, diagonal: calibrated) and expected utility versus ε (right). The hard rule’s implicit risk is 5–14%.

Table 4: Recourse: local repair versus full replanning over 20 DES realizations.

N	Fail/real	t_{local} (ms)	t_{full} (s)	Speedup	Recovery
6	0.15	2.1	0.690	334×	1.00
10	0.05	0.6	0.672	1065×	1.00

the nominal plan violates 29–45%. The right panel prices the guarantee: tightening ε from 0.5 (which behaviorally equals hard) to 0.05 cuts delivered violation from 5–14% to $< 1\%$ at a utility cost of roughly 1.2–1.5 $\times$. Equivalently, hard’s $\sim 10\%$ mean implicit risk is not free—it is a hidden reliability debt that the chance-constrained operator can buy down explicitly with a single scalar ε . Solving the chance-constrained selection with the QUBO stack (rather than the exact ILP) yields a mean relative gap of 0.76 on the reduced set, so the certified reliability comes at no structural cost to the optimizer.

7.7 Recourse and Optimality Certification

Table 4 shows recourse results over 20 DES realizations. Local repair recovers *all* failed requests (recovery rate 1.0) while costing 0.6–2.1 ms versus 0.67–0.69 s for full replanning—a 334–1065 $\times$ speedup—and recovers 35.5% of the utility lost to failures. The optimality benchmark (Fig. 14) certifies the solvers against exact ILP: Metropolis attains a mean relative gap of 1.5% (zero on most instances), SQA 46.1%, and the raw QUBO/hybrid at a small read budget 47.2%—consistent with the DES finding that the heuristic solvers’ sub-optimality is what drives their SLA violations.

7.8 Purification

Table 5 and Figure 15 show the purification comparison. On the tested chains the solver selects purification-free bundles in all three regimes (mean rounds $k = 0$), so a

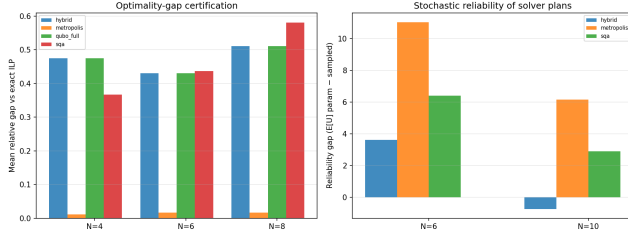


Figure 14: Optimality certification: mean relative gap versus exact ILP (left) and reliability gap of solver plans (right).

Table 5: Purification-aware versus agnostic candidate sets ($q_{\max} = 4$). The aware set is a superset; identical optimal selections confirm purification is cost-free when not binding.

N	Regime	Bundles	Utility	Served	Time (s)
6	agnostic	4	300.1	0.67	0.46
6	default $q \in [0..2]$	15	300.1	0.67	0.68
6	aware	27	300.1	0.67	0.75
10	agnostic	5	423.3	0.50	0.46
10	default $q \in [0..2]$	19	423.3	0.50	0.68
10	aware	39	423.3	0.50	0.81

purification-aware candidate superset is *costless*: utility, served ratio, and fidelity are identical while the candidate pool grows from 4 to 27 bundles ($N=6$). The fidelity sweep (Fig. 15, right) shows when purification binds: a single round lifts a 3-link path from $F = 0.634$ to $F = 0.703$, meeting a $F_{\min} = 0.7$ requirement that the $q=0$ bundle cannot satisfy—at the price of success probability 0.145 versus 0.512 and doubled resource use. Purification is therefore a Pareto trade-off surface, not a free fidelity multiplier, and treating it as a first-class optimization variable lets the solver price that trade-off exactly.

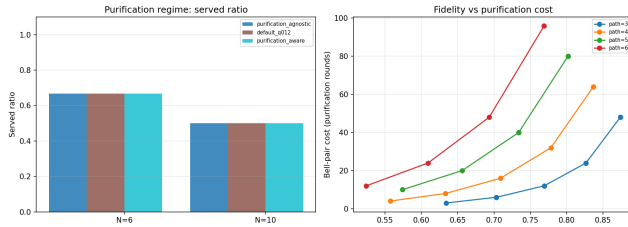


Figure 15: Purification as a first-class variable: regime comparison (left) and fidelity/utility sweep over purification rounds and path length (right).

Table 6: Adaptive-budget study: utility by budget k and the adaptive budget versus the full-QUBO reference.

N	$k=2$	$k=4$	$k=8$	Adaptive	Full-QUBO
8	11.20	11.20	11.20	11.20	12.83
12	11.20	11.20	11.20	11.20	12.91

Table 7: Embedded quantum annealing (QA, PIA) versus SA and SQA on identical QUBOs. QA reports hardware-qubit counts and chain-break fractions.

N	QA util	SA util	SQA util	Qubits	Chain breaks
4	5.11	0.68	2.92	832	0.00
6	6.15	0.68	5.52	832	0.00
8	11.45	0.68	2.86	896	0.00

7.9 Adaptive Budget and Quantum-Annealing Backend

Table 6 shows the adaptive-budget study: on both request counts every tested budget $k \geq 2$ —and the congestion-driven adaptive budget—returns the *same* utility (11.20), confirming that k is a free control knob until it discards the optimal bundle; the consistent $\sim 13\%$ gap versus the full-candidate reference is a fixed-read annealing artifact that candidate reduction does not worsen. Table 7 reports the embedded quantum-annealing backend. The embedded QA backend (832–896 hardware qubits, zero chain breaks) beats both classical samplers at every size (5.11–11.45 vs. 0.68 for SA, which stagnates at the small default read budget, and 2.86–5.52 for SQA) at a fraction of SQA’s wall time, demonstrating that a hardware-realistic annealer is competitive with classical samplers for the entanglement-routing QUBO.

7.10 Multi-Objective, Disjoint Paths, Swapping Order

Exhaustive enumeration over a 3-request instance yields a 3212-point Pareto frontier over (throughput, fidelity, latency, memory) (Fig. 16, left); the ϵ -constraint curves are flat in the tested range because the lightly-loaded optimizer has slack in both objectives. k -disjoint provisioning (Fig. 17) adds utility without reducing coverage: on $N=8$ both candidate sets serve 7/8 requests, but the multipath-enabled set raises aggregate utility from 460.9 to 462.5. Swap-tree ordering (Fig. 18) is fidelity- identical noiselessly at $L=3$; as L grows the balanced/optimal trees’ shorter hold times win, recovering up to $1.58\times$ of delivered fidelity at intermediate coherence times ($L=10$, $\tau_{\text{mem}} = 5$).

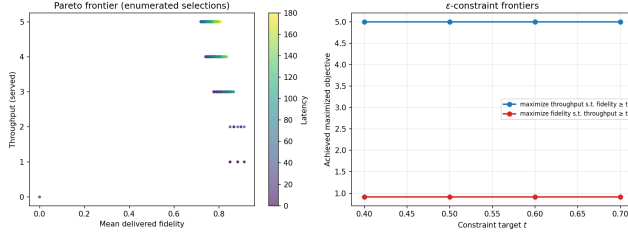


Figure 16: Pareto frontier over throughput/fidelity/latency (left) and ϵ -constraint frontiers (right).

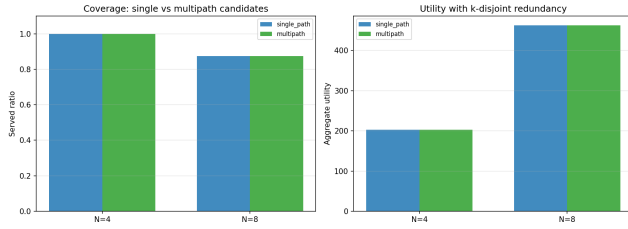


Figure 17: Coverage (left) and utility (right) of single-path versus k -disjoint multipath candidate sets.

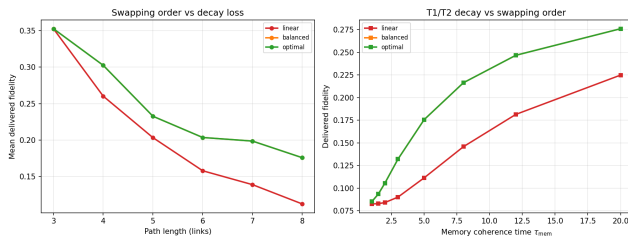


Figure 18: Delivered fidelity of linear, balanced, and optimal swap trees versus path length (left) and memory coherence time (right).

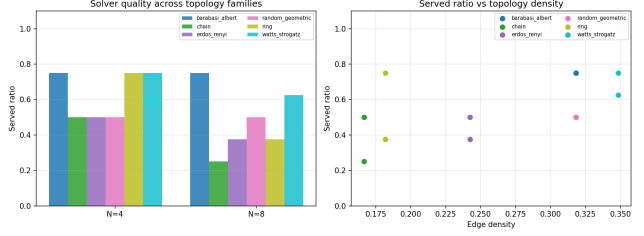


Figure 19: Served ratio by topology family (left) and versus edge density (right) for five generative network families.

7.11 Topology Robustness

Figure 19 runs the solver on all five generative families. Watts–Strogatz small-world topologies serve the full request set at both $N=4$ and $N=8$; Erdős–Rényi (0.75/0.625) and ring/chain (0.5/0.625) are comparable; random-geometric (0.5/0.375) and Barabási–Albert (0.25/0.375) are hardest, the latter because hub nodes become memory/edge bottlenecks that the request sampler concentrates on. Served ratio correlates positively with edge density and minimum degree.

7.12 Prior-Extension Recap

Four extensions close our prior future-work list. (i) A *time-dependent decoherence-aware router* lifts delivered fidelity from 0.188 to 0.193 (+2.5%) on a Poisson trace at equal served count by aligning the risk term’s hold model with the evaluation decay law. (ii) A *hard-constraint MPS* matches penalty-MPS served ratio while being 1.4–1.7 $\times$ faster, with feasibility guaranteed by construction. (iii) A tabular *Q-learning router* matches the streaming annealer’s utility (189.5, 6/7 served) with 0.2ms inference versus 5.3ms. (iv) *Hardware profiles*—ideal, photonic/fiber, ion-trap, and superconducting—reveal a 2.8 $\times$ delivered-fidelity spread (0.761 down to 0.269), quantifying when coherence-aware scheduling is indispensable.

8 Discussion

Deterministic fidelity constraints are an uncalibrated reliability claim. The central methodological finding of this paper is that the standard rule $F_{r,b} \geq F_{\min}^{(r)}$, ubiquitous in entanglement-routing work, silently tolerates a violation probability up to 1/2 for any bundle whose nominal fidelity sits at the threshold—and the empirical mean is 5–14% even when the solver selects high-quality plans. The chance-constrained quantile rule (Eq. (20)) replaces this implicit, unmeasured risk with an explicit, validated budget ϵ , and our DES study confirms the empirical violation stays at or below

ε . We recommend that routing papers report *parametric and sampled SLA jointly*, as we do in Table 3, so that throughput is never reported without its fidelity cost.

The utility price of reliability is real but controlled. Buying the guarantee from hard’s implicit $\sim 10\%$ down to $\varepsilon = 0.05$ costs roughly $1.2\text{--}1.5\times$ in utility because marginal (high-risk) bundles are refused. This is the correct economic statement: the operator trades capacity for certified reliability with a single knob ε , and the frontier in Fig. 13 (right) is the explicit menu. Notably, the chance-constrained set is no harder to optimize than the hard set (mean QUBO gap 0.76 on the reduced set), so the reliability guarantee does not tax the solver.

Noise and decoherence are separable failure modes. The DES separates *noise* (Gaussian fluctuations of delivered fidelity) from *decoherence* (deterministic decay during storage). Chance constraints calibrate the former; the temporal scheduler (Sec. 4.5) and swap-tree ordering (Sec. 6.3) address the latter; recourse (Sec. 7.7) mitigates residual stochastic failure. A plan that is optimal under one model need not be under the other—the SQA plan’s up-to-8% SLA violations (Table 3) are exactly the price of its 46% mean optimality gap (Fig. 14). Reliability analysis is therefore not a post-processing nicety but a first-class objective.

Penalty tightening is regime-dependent. Reachable-load calibration changes the sufficient QUBO certificate, not the constrained objective. As request count grows, $\delta = 1$ becomes more common and tightening rarer; this association is consistent with the proposed mechanism but is not causal evidence. Because every matched interval includes zero, the experiment neither establishes solver equivalence nor detects systematic improvement under the tested read budgets. Fixing A isolates capacity calibration; joint scaling and hardware-range effects remain open.

Candidate reduction and purification are free until they bind. Adaptive top- k , GNN-guided ranking, and purification-aware candidate supersets all leave the reference optimum unchanged on our benchmarks—reduction improves anneal quality at fixed budget, and purification is never forced. Each becomes binding in a defined regime (budget $k = 2$ on chains; $F_{\min}$ above the $q=0$ ceiling), which the corresponding sweep quantifies. This reframes both as *tunable reliability/capacity dials* rather than approximations.

Limitations. The MPS coupling order is heuristic; a learned ordering could improve bond efficiency. The

GNN ranker is trained per instance family and not yet shown to generalize across topologies. The joint scheduler assumes deterministic transmission; stochastic success would require probabilistic windows. The truncated-normal noise model is a proxy for a full device-noise characterization, and the chance-constraint calibration holds for the modeled noise only—hardware validation on measured parameters remains future work. Our swap-tree analysis assumes independent T1/T2 decay without correlated dephasing.

9 Conclusion

We presented a QUBO-based framework for entanglement routing whose solver stack (Metropolis, boundary-MPS, OpenJij SA, embedded QA) is certified against exact ILP and extended from static, memory-blind routing toward the stochastic, temporal, and learned regime that deployed quantum networks will require. The key findings are: (i) solver families agree on the capacity-limited served-ratio frontier, with the MPS solver $5\text{--}4,000\times$ faster and Metropolis within 1.5% of the exact ILP; (ii) the direct slack-free Hamiltonian dominates the slack-variable QUBO in size and quality, while the request-conditioned reachable-load calibration preserves exact ground-state correspondence, tightens at least one capacity coefficient in 12.9% of benchmark instances, and yields no detected systematic SA/SQA difference under the tested read budget; (iii) temporal-memory joint scheduling delivers every admitted request within coherence at fidelity 0.30–0.51, where a memory-agnostic baseline over-admits; (iv) chance-constrained routing turns the implicit 5–14% SLA risk of the deterministic rule into a validated, tunable ε , with the empirical violation at or below budget on every tested instance; (v) recourse recovers 100% of failures at $334\text{--}1065\times$ speedup; (vi) candidate reduction, purification, and k -disjoint provisioning are free-or-beneficial until defined regimes make them bind; (vii) balanced swap trees recover up to $1.58\times$ of delivered fidelity under memory decay; (viii) solver quality survives across five topology families, degrading gracefully only on scale-free networks.

Remaining future work includes: adaptive bond dimension driven by truncation error; function-approximation RL that generalizes across topologies; learned MPS coupling order; stochastic-window joint scheduling; hardware validation of the noise model on measured testbed parameters; and chance-constrained joint scheduling that co-optimizes ε with the temporal admission decision.

References

- [1] H. J. Kimble, “The quantum internet,” *Nature*, vol. 453, pp. 1023–1030, 2008.
- [2] S. Wehner, D. Elkouss, and R. Hanson, “Quantum internet: A vision for the road ahead,” *Science*, vol. 362, no. 6412, p. eaam9288, 2018.
- [3] C. Simon, “Towards a global quantum network,” *Nat. Photonics*, vol. 11, pp. 678–680, 2017.
- [4] H.-J. Briegel, W. Dür, J. I. Cirac, and P. Zoller, “Quantum repeaters: The role of imperfect local operations in quantum communication,” *Phys. Rev. Lett.*, vol. 81, p. 5932, 1998.
- [5] W. Dür, H.-J. Briegel, J. I. Cirac, and P. Zoller, “Quantum repeaters based on entanglement purification,” *Phys. Rev. A*, vol. 59, p. 169, 1999.
- [6] N. Sangouard, C. Simon, H. de Riedmatten, and N. Gisin, “Quantum repeaters based on atomic ensembles and linear optics,” *Rev. Mod. Phys.*, vol. 83, p. 33, 2011.
- [7] K. Azuma, S. E. Economou, D. Elkouss, P. Hilaire, L. Jiang, H.-K. Lo, and I. Tzitrin, “Quantum repeaters: From quantum networks to the quantum internet,” *Rev. Mod. Phys.*, vol. 95, p. 045006, 2023.
- [8] R. Horodecki, P. Horodecki, M. Horodecki, and K. Horodecki, “Quantum entanglement,” *Rev. Mod. Phys.*, vol. 81, pp. 865–942, 2009.
- [9] R. F. Werner, “Quantum states with Einstein–Podolsky–Rosen correlations admitting a hidden-variable model,” *Phys. Rev. A*, vol. 40, pp. 4277–4281, 1989.
- [10] R. Jozsa, “Fidelity for mixed quantum states,” *J. Mod. Opt.*, vol. 41, no. 12, pp. 2315–2323, 1994.
- [11] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, “Teleporting an unknown quantum state via dual classical and Einstein–Podolsky–Rosen channels,” *Phys. Rev. Lett.*, vol. 70, p. 1895, 1993.
- [12] C. H. Bennett, G. Brassard, S. Popescu, B. Schumacher, J. A. Smolin, and W. K. Wootters, “Purification of noisy entanglement and faithful teleportation via noisy channels,” *Phys. Rev. Lett.*, vol. 76, p. 722, 1996.
- [13] A. K. Ekert, “Quantum cryptography based on Bell’s theorem,” *Phys. Rev. Lett.*, vol. 67, p. 661, 1991.
- [14] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, “Quantum cryptography,” *Rev. Mod. Phys.*, vol. 74, pp. 145–195, 2002.
- [15] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, “The security of practical quantum key distribution,” *Rev. Mod. Phys.*, vol. 81, pp. 1301–1350, 2009.
- [16] S. Pirandola, U. L. Andersen, L. Banchi, M. Berta, D. Bunandar, R. Colbeck, D. Englund, T. Gehring, C. Lupo, C. Ottaviani, J. L. Pereira, M. Razavi, J. S. Shaari, M. Tomamichel, V. C. Usenko, G. Vallone, P. Villoresi, and P. Wallden, “Advances in quantum cryptography,” *Adv. Opt. Photon.*, vol. 12, p. 1012, 2020.
- [17] R. Van Meter, *Quantum Networking*. Wiley, 2014.
- [18] M. Pant, H. Krovi, D. Towsley, L. Tassiulas, L. Jiang, P. Basu, D. Englund, and S. Guha, “Routing entanglement in the quantum internet,” *npj Quantum Information*, vol. 5, p. 25, 2019.
- [19] M. Caleffi, “Optimal routing for quantum networks,” *IEEE Access*, vol. 5, pp. 22299–22312, 2017.
- [20] L. Gyongyosi and S. Imre, “Entanglement-gradient routing of quantum networks,” *Sci. Rep.*, vol. 9, p. 19274, 2019.
- [21] M. Cuquet and J. Calsamiglia, “Entanglement percolation in quantum complex networks,” *Phys. Rev. Lett.*, vol. 103, p. 240503, 2009.
- [22] E. Schoute, L. Mancinska, T. Islam, A. Kahanamoku-Meyer, and S. Wehner, “Shortcuts to quantum network routing,” arXiv:1610.05238, 2016.
- [23] W. Kozłowski, A. Dahlberg, and S. Wehner, “Designing a quantum network protocol,” in *Proc. ACM CoNEXT*, 2020.
- [24] T. Coopmans, R. Knegjens, A. Dahlberg, D. Maier, L. Nijsten, A. N. de Oliveira, M. Papendrecht, J. Rabbie, F. Rozpędek, M. Skrzypczyk, L. Wubben, W. de Jong, D. Podareanu, A. Torres-Knoop, D. Elkouss, and S. Wehner, “NetSquid, a NETwork SIMulator for Quantum Information using Discrete events,” *Commun. Phys.*, vol. 4, p. 164, 2021.
- [25] X. Wu, A. Kolar, J. Chung, D. Jin, T. Zhong, R. Kettimuthu, and M. Suchara, “SeQUeNCe: A customizable discrete-event simulator of quantum networks,” *Quantum Sci. Technol.*, vol. 6, p. 045027, 2021.

- [26] T. Kadowaki and H. Nishimori, “Quantum annealing in the transverse Ising model,” *Phys. Rev. E*, vol. 58, p. 5355, 1998.
- [27] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, p. 671, 1983.
- [28] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose, “Quantum annealing with manufactured spins,” *Nature*, vol. 473, pp. 194–198, 2011.
- [29] S. Boixo, T. F. Rønnow, S. V. Isakov, Z. Wang, D. Wecker, D. A. Lidar, J. M. Martinis, and M. Troyer, “Evidence for quantum annealing with more than one hundred qubits,” *Nat. Phys.*, vol. 10, pp. 218–224, 2014.
- [30] S. E. Venegas-Andraca, W. Cruz-Santos, C. McGeoch, and M. Lanzagorta, “A cross-disciplinary glimpse at practical quantum annealing,” *Int. J. Circuit Theory Appl.*, vol. 47, pp. 853–883, 2019.
- [31] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, p. 5, 2014.
- [32] F. Glover, G. Kochenberger, and Y. Du, “A tutorial on formulating and using QUBO models,” *4OR*, vol. 17, pp. 335–369, 2019.
- [33] A. Verma and M. Lewis, “Penalty and partitioning techniques to improve performance of QUBO solvers,” *Discrete Optimization*, vol. 44, p. 100594, 2022.
- [34] E. Alessandroni, S. Ramos-Calderer, I. Roth, E. Traversi, and L. Aolita, “Alleviating the quantum Big-M problem,” *npj Quantum Information*, vol. 11, p. 125, 2025.
- [35] R. A. Quintero and L. F. Zuluaga, “Characterizing and benchmarking QUBO reformulations of the knapsack problem,” Lehigh University ISE Technical Report 21T-028, 2021.
- [36] E. Güney, J. Ehrental, and T. Hanne, “QUBO formulations and characterization of penalty parameters for the multi-knapsack problem,” *IEEE Access*, vol. 13, pp. 47086–47098, 2025.
- [37] M. Díez García, M. Ayodele, and A. Moraglio, “Exact and sequential penalty weights in quadratic unconstrained binary optimisation with a Digital Annealer,” in *Proc. GECCO Companion*, pp. 184–187, 2022.
- [38] M. Ayodele, “Penalty weights in QUBO formulations: Permutation problems,” in *Evolutionary Computation in Combinatorial Optimization*, Lecture Notes in Computer Science, vol. 13222, pp. 159–174, Springer, 2022.
- [39] J. Liu and A. Moraglio, “A framework for automatically setting multiple penalty weights in quadratic unconstrained binary optimization,” in *Proc. GECCO Companion*, pp. 575–578, 2025.
- [40] J. Liu and A. Moraglio, “Structure-aware penalty weights scaling for quadratic unconstrained binary optimisation formulation,” in *Applications of Evolutionary Computation*, Lecture Notes in Computer Science, vol. 16525, pp. 457–472, Springer, 2026.
- [41] O. Şeker, M. Bodur, and H. Pouya, “Routing and wavelength assignment with protection: A quadratic unconstrained binary optimization approach enabled by Digital Annealer technology,” *IIEE Transactions*, vol. 56, no. 2, pp. 156–171, 2024.
- [42] P. Sinha and A. A. Zoltners, “The multiple-choice knapsack problem,” *Operations Research*, vol. 27, no. 3, pp. 503–515, 1979.
- [43] M. A. Trick, “A dynamic programming approach for consistency and propagation for knapsack constraints,” *Annals of Operations Research*, vol. 118, pp. 73–84, 2003.
- [44] L. Perron, F. Didier, and S. Gay, “The CP-SAT-LP solver,” in *Proc. 29th International Conference on Principles and Practice of Constraint Programming*, LIPIcs, vol. 280, pp. 3:1–3:2, 2023.
- [45] K. Tanahashi, S. Takayanagi, T. Motohashi, and S. Tanaka, “Application of Ising machines and a software development methodology to portfolio optimization,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 5, pp. 2604–2616, 2022.
- [46] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” arXiv:1411.4028, 2014.
- [47] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [48] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, “Equation of state calculations by fast computing machines,” *J. Chem. Phys.*, vol. 21, p. 1087, 1953.
- [49] OpenJij: An open-source framework for quantum and simulated annealing. <https://openjij.github.io/>

- [50] S. R. White, “Density matrix formulation for quantum renormalization groups,” *Phys. Rev. Lett.*, vol. 69, p. 2863, 1992.
- [51] G. Vidal, “Efficient classical simulation of slightly entangled quantum computations,” *Phys. Rev. Lett.*, vol. 91, p. 147902, 2003.
- [52] U. Schollwöck, “The density-matrix renormalization group in the age of matrix product states,” *Ann. Phys.*, vol. 326, pp. 96–192, 2011.
- [53] R. Orús, “A practical introduction to tensor networks: Matrix product states and projected entangled pair states,” *Ann. Phys.*, vol. 349, pp. 117–158, 2014.
- [54] R. Orús, “Tensor networks for complex quantum systems,” *Nat. Rev. Phys.*, vol. 1, pp. 538–550, 2019.
- [55] T. Hao, X. Huang, R. Jia, and C. Peng, “A quantum-inspired tensor network algorithm for constrained combinatorial optimization problems,” *Frontiers in Physics*, vol. 10, 2022, arXiv:2203.15246.
- [56] K. Nakada, K. Tanahashi, and S. Tanaka, “Quick design of feasible tensor networks for constrained combinatorial optimization,” *Quantum*, vol. 9, p. 1799, 2025.
- [57] F. Preisser, O. Mc Keever, and M. Lubasch, “Variational matrix product states for combinatorial optimization,” *Phys. Rev. Research*, vol. 8, p. 033027, 2026.
- [58] A. Mata Ali, “Explicit solution equation for every combinatorial problem via tensor networks: MeLo-CoToN,” arXiv:2502.05981, 2025.
- [59] A. Mata Ali, L. Delgado, A. Roura, and I. de Leceta, “Polynomial-time solver of tridiagonal QUBO, QUDO and tensor QUDO problems with tensor networks,” arXiv:2309.10509, 2023.
- [60] F. Rosales, “Quantum-inspired optimization for adaptive multi-demand routing in QKD networks,” arXiv:2605.27425, 2025.
- [61] “Q-Learning for Resource-Aware and Adaptive Routing in Trusted-Relay QKD Networks,” 2025.
- [62] J. A. Boyan and M. Littman, “Packet routing in dynamically changing networks: A reinforcement learning approach,” in *Proc. NeurIPS*, 1993.
- [63] B. Mao, F. Fadlullah, N. Kato, F. Tang, and T. Onouchi, “Routing or computing? The paradigm shift towards intelligent computer network packet transmission based on deep learning,” *IEEE Trans. Computers*, vol. 66, pp. 1946–1960, 2017.
- [64] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, “RouteNet: Learning optimized routes with traffic flows as messages,” *IEEE J. Sel. Areas Commun.*, vol. 38, pp. 192–205, 2020.
- [65] E. Koutsoupias and C. Papadimitriou, “Worst-case equilibria,” in *Proc. STACS*, 1999, pp. 404–413.
- [66] T. Roughgarden and E. Tardos, “How bad is selfish routing?” *J. ACM*, vol. 49, pp. 236–259, 2002.
- [67] D. Braess, “Über ein Paradoxon aus der Verkehrsplanung,” *Unternehmensforschung*, vol. 12, pp. 258–268, 1968.
- [68] D. Bertsimas and M. Sim, “The price of robustness,” *Operations Research*, vol. 52, pp. 35–53, 2004.
- [69] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001.
- [70] A. Charnes and W. W. Cooper, “Chance-constrained programming,” *Management Science*, vol. 6, no. 1, pp. 73–79, 1959.
- [71] A. Prékopa, *Stochastic Programming*. Kluwer, 1995.
- [72] J. R. Birge and F. Louveaux, *Introduction to Stochastic Programming*. Springer, 1997.
- [73] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*. Princeton University Press, 2009.
- [74] A. I. Lvovsky, B. C. Sanders, and W. Tittel, “Optical quantum memory,” *Nat. Photonics*, vol. 3, pp. 706–714, 2009.